

CISP 400

Object Oriented Programming in C++

Dr. Fowler



F O L S O M L A K E C O L L E G E
EL DORADO CENTER | RANCHO CORDOVA CENTER

AN x64 PROCESSOR IS SCREAMING ALONG AT BILLIONS OF CYCLES PER SECOND TO RUN THE XNU KERNEL, WHICH IS FRANTICALLY WORKING THROUGH ALL THE POSIX-SPECIFIED ABSTRACTION TO CREATE THE DARWIN SYSTEM UNDERLYING OS X, WHICH IN TURN IS STRAINING ITSELF TO RUN FIREFOX AND ITS GECKO RENDERER, WHICH CREATES A FLASH OBJECT WHICH RENDERS DOZENS OF VIDEO FRAMES EVERY SECOND

BECAUSE I WANTED TO SEE A CAT JUMP INTO A BOX AND FALL OVER.



I AM A GOD.

Last Time

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Theme for Today

“Take a breather”

OOP is about classes interacting

Date1.cpp

Date2.cpp

Date3.cpp

Anonymous classes

Sometimes we don't want to incur the overhead of constructing an object, and having it survive for significant amounts of time.

Anonymous Classes

- Does not have a **class** name.
- **Cannot** be used in functions/methods.
- Has a variable name, though. So we can call it. Has scope.

```
1  #include <iostream>
2  using namespace std;
3
4  class {
5  public:
6      int x, y;
7  } point1, point2;
8
9  int main() {
10     point1.x = 10;
11     point2.y = 23;
12
13     return 0;
14 }
```

Anonymous objects

Sometimes we don't want to incur the overhead of constructing an object, and having it survive for **any** length of time.

```
1  #include <iostream>
2  using namespace std;
3
4  class Printer {
5  public:
6      Printer(string msg = "NA") {
7          cout << msg << std::endl;
8      }
9  };
10
11  int main() {
12      // Temporary/anonymous object - no variable name
13      // Create, use, destroy immediately
14      Printer("Hello!");
15
16      return 0;
17  }
```

```
1  #include <iostream>
2  using namespace std;
3
4  class Printer {
5  public:
6      Printer(string msg = "NA") {
7          cout << msg << std::endl;
8      }
9  };
10
11  int main() {
12      // Temporary/anonymous object - no variable name
13      // Create, use, destroy immediately
14      Printer("Hello!");
15
16      return 0;
17  }
```

Hello!

=== Code Execution Successful ===

Anonymous Object

- Does not have a **variable** name.
- Cannot make multiple instances of specific object.
- **Can** be used in functions/methods.
- We use it where we need it.

Use Case

- Assume we create a whizbang class which performs some amazing processing to multimedia files.
- To use it, we just send the multimedia file to our whizbang object.
- Knowing what we know about objects, what does this mean? Hint: How many different multimedia file types are there out there?


```
1  #include <iostream>
2  using namespace std;
3
4  class Whizbang {
5  public:
6      Whizbang(pdf in&) {
7          // process multimedia file
8      }
9      Whizbang(mp3 in&) {
10         // process multimedia file
11     }
12     // AVI, JPG, PNG, etc. etc.
13 };
14
15 int main() {
16     PDF myPDF;          // supplier objects
17     MP3 myMP3;
18     Whizbang processPDF(myPDF); // client objects
19     Whizbang processMP3(myMP3);
20
21     return 0;
22 }
```

Adapters

Adapter

- Bridges the gap between one object format and another.
- Usually temporary, often 1 shot.



A Pattern Language

Towns · Buildings · Construction



Christopher Alexander

Sara Ishikawa · Murray Silverstein

WITH

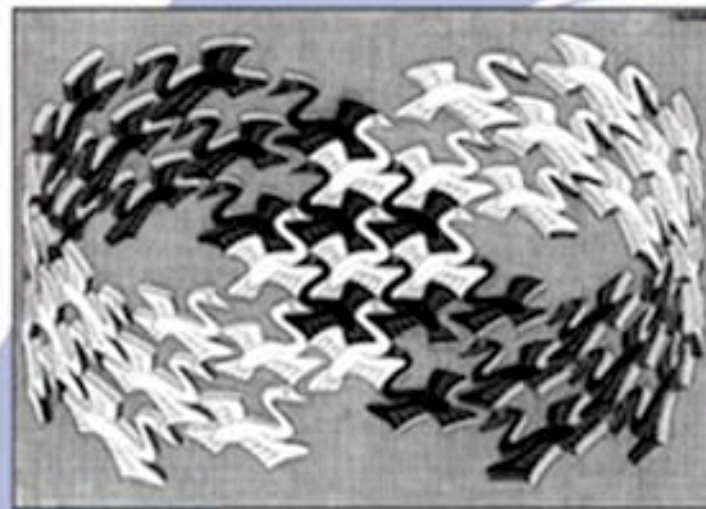
Max Jacobson · Ingrid Fiksdahl-King

Shlomo Angel

Design Patterns

Elements of Reusable Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch

ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES



Creational Patterns

1. Abstract Factory
2. Builder
3. Factory Method
4. Prototype
5. Singleton

Structural Patterns

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Façade
6. Flyweight
7. Proxy

Behavioral Patterns

1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
7. Observer
8. State
9. Strategy
10. Template Method
11. Visitor

Gang of Four (GoF) Design Patterns

Creational Patterns

1. Abstract Factory
2. Builder
3. Factory Method
4. Prototype
5. Singleton

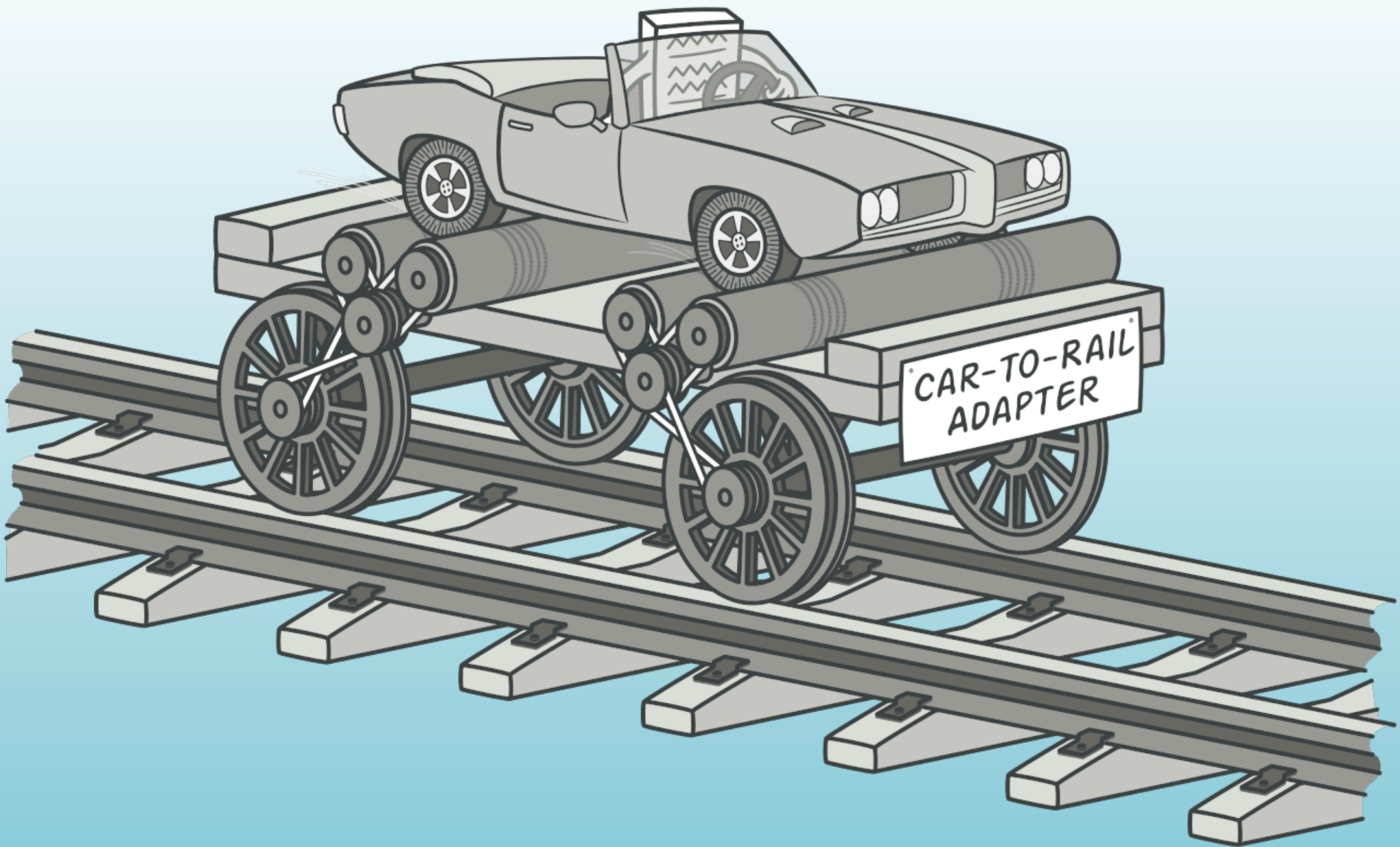
Structural Patterns

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Façade
6. Flyweight
7. Proxy

Behavioral Patterns

1. Chain of Responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
7. Observer
8. State
9. Strategy
10. Template Method
11. Visitor

Gang of Four (GoF) Design Patterns



<https://refactoring.guru/design-patterns/adapter>

Back to Whizbang

- We no longer need a ton of different multimedia types built in.
- Instead, we accept one relatively broad type.
- Then we convert everything to that type.

```
1  #include <iostream>
2  using namespace std;
3
4  class Whizbang {
5  public:
6      Whizbang(string in&) {
7          // process multimedia file
8      }
9  };
10 class PDF2String {
11     public:
12     PDF2String(PDF in&){
13         cout << // convert to string << endl;
14     }
15 }
16
17 int main() {
18     PDF myPDF;          // supplier objects
19     MP3 myMP3;
20     Whizbang processPDF(PDF2String(myPDF));
21     Whizbang processMP3(MP32String(myMP3)); // not shown
22
23     return 0;
24 }
```

```
1  #include <iostream>
2  using namespace std;
3
4  class Whizbang {
5  public:
6      Whizbang(string in&) {
7          // process multimedia file
8      }
9  };
10 class {
11     public:
12     PDF2String(PDF in&){
13         cout << // convert to string << endl;
14     }
15 } PDF2S(myPDF);
16
17 int main() {
18     PDF myPDF;          // supplier objects
19     MP3 myMP3;
20     Whizbang processPDF(PDF2S);
21     Whizbang processMP3(MP32S); // not shown
22
23     return 0;
24 }
```

```
1  #include <iostream>
2  using namespace std;
3
4  class Whizbang {
5  public:
6      Whizbang(string in&) {
7          // process multimedia file
8      }
9  };
10 class PDF2String{
11     public:
12     PDF2String(PDF in&){
13         cout << // convert to string << endl;
14     }
15 };
16
17 int main() {
18     PDF myPDF;          // supplier objects
19     MP3 myMP3;
20     PDF2String PDF2S(myPDF);
21     Whizbang processPDF(PDF2S);
22     Whizbang processMP3(MP32S); // not shown
23
24     return 0;
25 }
```



What's this section
about?

- Learning OOP is about objects working together.



NEXT TIME

Ch 11 - More OOP

11.1 The this pointer and Constant member Functions

11.2 Static Members

11.3 Friends of Classes

11.4 Memberwise Assignment

11.5 Copy Constructors

11.6 Operator Overloading

11.7 Rvalue References and Move Operations

11.8 Function Objects and Lambda Expressions

11.9 Type Conversion Operators

11.10 Convergent Constructors

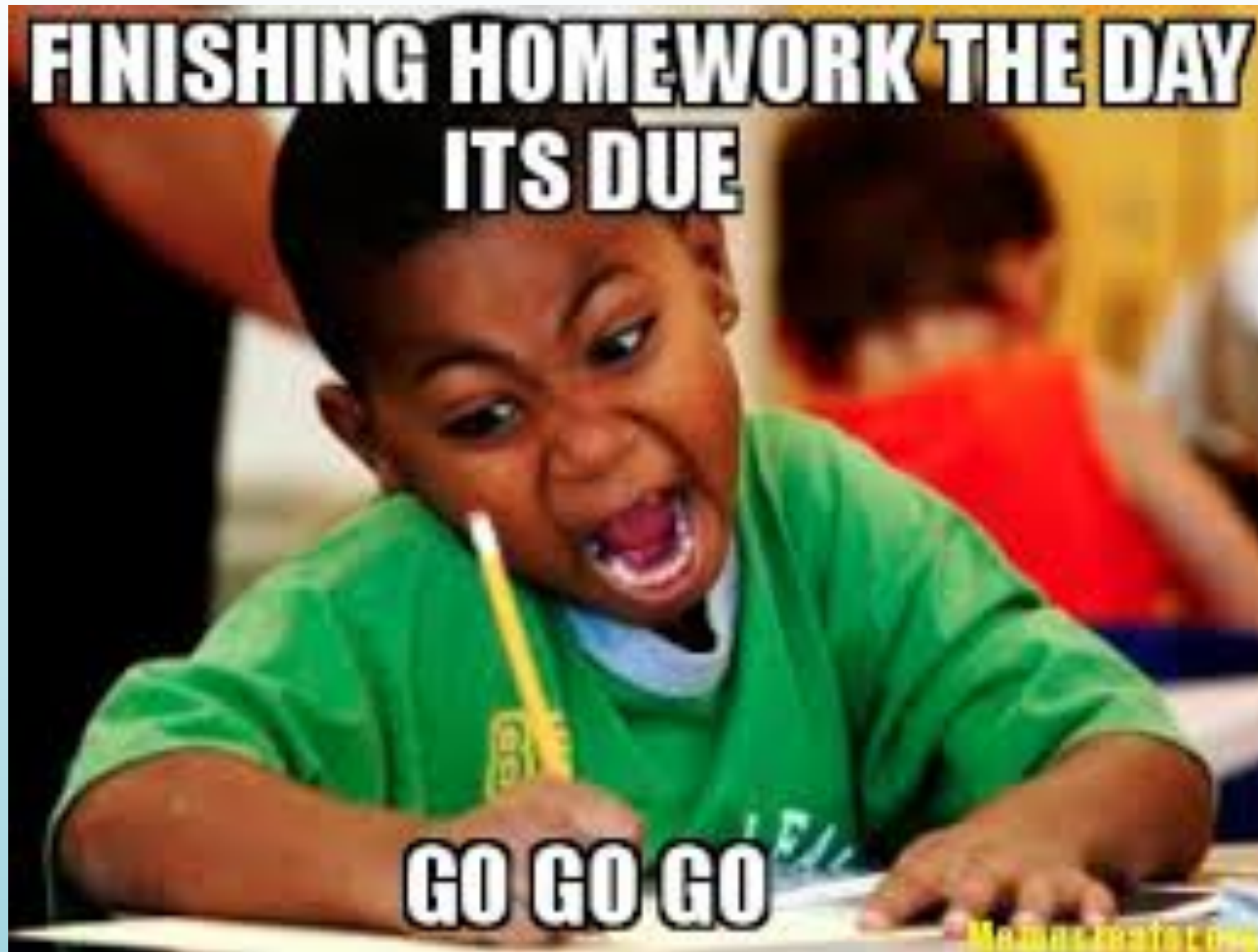
11.11 Aggregation and Composition

11.12 Inheritance

11.13 Protected Members and Class Access

11.14 Constructors, Destructors, and Inheritance

11.15 Overriding Base Class Functions



Homework 2 Peer Review due Sunday.

**HOW ARE WE
SUPPOSED TO SAMPLE THIS?**

**WHY IS SHE
WEARING GLOVES?**

Homework Preflight

Hw3 - InvInq

Exercise Goals

- Go over the homework and check understanding.
- Get questions answered.
- Discuss potential designs.

Task

- Read the next homework due in Canvas.
- Write down any questions you have about it.

Task - II

- Pair Up
- Go over each other's questions & see if you can answer them.

Task - III

- Ask me any unanswered questions.

Class Starters

Exercise

What's the difference ...

- `char* = "This is a test";`
`char a[] = "This is a test\0";`

Sentence Starters

- The question I'd like most to ask the author is...
- The idea I most take issue with from the text is ...
- The most critical point from last week's class discussion was ...
- The idea I found most interesting was ...
- The part of the class session I found most confusing was ...
- The part of the class session which surprised me the most was ...
- The idea that helped me relate the class session to what I already know about the topic was ...